

Resumen Teórico

Introducción a la Estructura de Datos

Material de Estudio para Trabajo Final Integrador

Mayo 2026

1. Unidad 1: Modelos Matemáticos y Teoría de Grafos

1.1. Conceptos Básicos de Grafos

1.1.1. Definición

Un **grafo** $G = (V, E)$ está compuesto por:

- V : Conjunto finito no vacío de **vértices** o nodos
- E : Conjunto de **aristas** o arcos que conectan pares de vértices

1.1.2. Tipos de Grafos

Grafo Dirigido (Dígrafo): Las aristas tienen dirección. Se representa como (v_i, v_j) donde la arista va desde v_i hacia v_j .

Grafo No Dirigido: Las aristas no tienen dirección. La arista $\{v_i, v_j\}$ conecta ambos vértices bidireccionalmente.

1.1.3. Propiedades de Grafos

Conexidad:

- **Grafo Conexo:** Existe un camino entre cualquier par de vértices.
- **Grafo No Conexo:** Existen vértices sin camino entre ellos.
- **Componente Conexa:** Subgrafo conexo maximal.

Grafo Ponderado: Un grafo donde cada arista tiene asociado un peso o costo numérico.

$$w : E \rightarrow \mathbb{R}$$

Grafo Maximal y Minimal:

- **Maximal:** No se pueden agregar más aristas sin violar alguna propiedad.
- **Minimal:** No se pueden quitar aristas sin violar alguna propiedad.

1.1.4. Representación de Grafos

Matriz de Adyacencia: Matriz A de tamaño $n \times n$ donde:

$$A[i][j] = \begin{cases} 1 & \text{si existe arista de } v_i \text{ a } v_j \text{ (no ponderado)} \\ w_{ij} & \text{peso de la arista (ponderado)} \\ 0 \text{ o } \infty & \text{si no existe arista} \end{cases}$$

Ejemplo: Para un grafo con 4 vértices $\{A, B, C, D\}$ con aristas $A - B$, $B - C$, $C - D$:

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Lista de Adyacencia: Para cada vértice, se mantiene una lista de sus vértices adyacentes.

Ejemplo:

A $\rightarrow$ [B]
B $\rightarrow$ [A, C]
C $\rightarrow$ [B, D]
D $\rightarrow$ [C]

1.2. Conceptos Adicionales

Grado de un Vértice:

- **Grafo no dirigido:** Número de aristas incidentes.
- **Grafo dirigido:**
 - *Grado de entrada:* Aristas que llegan al vértice
 - *Grado de salida:* Aristas que salen del vértice

Camino: Secuencia de vértices $v_1, v_2, \dots, v_k$ donde existe arista entre vértices consecutivos.

Ciclo: Camino donde el vértice inicial coincide con el final.

2. Unidad 2: Estructuras Lineales de Datos

2.1. Listas

2.1.1. Definición

Una **lista** es una colección ordenada de elementos del mismo tipo, donde cada elemento tiene una posición específica.

2.1.2. Tipos de Listas

Lista Secuencial: Los elementos se almacenan en posiciones contiguas de memoria (array).

Ventajas:

- Acceso directo por índice: $O(1)$
- Eficiente en uso de memoria

Desventajas:

- Inserción/eliminación costosa: $O(n)$
- Tamaño fijo

Lista Enlazada (Linkeada): Los elementos se almacenan en nodos, cada uno con un puntero al siguiente.

Estructura de un Nodo:

```
struct Nodo {  
    dato: TipoDato  
    siguiente: Puntero a Nodo  
}
```

Ventajas:

- Inserción/eliminación eficiente: $O(1)$ si se tiene el puntero
- Tamaño dinámico

Desventajas:

- Acceso secuencial: $O(n)$
- Mayor uso de memoria (punteros)

2.2. Pila (Stack)

2.2.1. Definición

Estructura LIFO (*Last In, First Out*): el último elemento en entrar es el primero en salir.

2.2.2. Operaciones Básicas

- `Push(x)`: Agregar elemento x al tope
- `Pop()`: Eliminar y retornar el elemento del tope
- `Top()` o `Peek()`: Consultar el elemento del tope sin eliminarlo
- `isEmpty()`: Verificar si la pila está vacía

2.2.3. Ejemplo Práctico

Operaciones:	Pila resultante:
<code>Push(5)</code>	[5]
<code>Push(3)</code>	[5, 3]
<code>Push(8)</code>	[5, 3, 8] <- Tope
<code>Pop()</code>	[5, 3] (retorna 8)
<code>Push(1)</code>	[5, 3, 1] <- Tope

2.3. Cola (Queue)

2.3.1. Definición

Estructura FIFO (*First In, First Out*): el primer elemento en entrar es el primero en salir.

2.3.2. Operaciones Básicas

- `Enqueue(x)` o `Insert(x)`: Agregar elemento x al final
- `Dequeue()` o `Remove()`: Eliminar y retornar el elemento del frente
- `Front()`: Consultar el elemento del frente sin eliminarlo
- `isEmpty()`: Verificar si la cola está vacía

2.3.3. Ejemplo Práctico

Operaciones:	Cola resultante:
<code>Enqueue(7)</code>	[7]
<code>Enqueue(3)</code>	[7, 3]
<code>Enqueue(9)</code>	[7, 3, 9]
<code>Dequeue()</code>	[3, 9] (retorna 7)
<code>Enqueue(2)</code>	[3, 9, 2]
	^Frente ^Final

2.4. Asignación Dinámica de Memoria

Concepto: Permite solicitar memoria durante la ejecución del programa según sea necesario.

Operaciones:

- `malloc()` / `new`: Reservar memoria
- `free()` / `delete`: Liberar memoria

2.5. Lista Irrestriccta

Definición: Lista donde se pueden realizar operaciones de inserción y eliminación en cualquier posición, no solo en los extremos.

Operaciones:

- Insertar en posición i
- Eliminar en posición i
- Buscar elemento
- Recorrer la lista

3. Unidad 3: Árboles

3.1. Definiciones Básicas

3.1.1. Árbol

Estructura jerárquica compuesta por nodos conectados por aristas, donde:

- Existe un nodo especial llamado **raíz**
- Cada nodo puede tener cero o más **hijos**
- No existen ciclos

3.1.2. Terminología

- **Raíz:** Nodo superior sin padre
- **Hoja:** Nodo sin hijos
- **Nodo Interno:** Nodo con al menos un hijo
- **Padre:** Nodo predecesor directo
- **Hijo:** Nodo sucesor directo
- **Hermanos:** Nodos con el mismo padre
- **Subárbol:** Árbol formado por un nodo y todos sus descendientes
- **Nivel:** Distancia desde la raíz (la raíz está en nivel 0)
- **Altura:** Máximo nivel del árbol (longitud del camino más largo desde la raíz a una hoja)
- **Profundidad de un nodo:** Longitud del camino desde la raíz hasta ese nodo

3.2. Árboles Binarios

3.2.1. Definición

Árbol donde cada nodo tiene **como máximo dos hijos**: hijo izquierdo e hijo derecho.

3.2.2. Tipos de Árboles Binarios

Árbol Binario Completo: Todos los niveles están completamente llenos, excepto posiblemente el último, que se llena de izquierda a derecha.

Árbol Binario Lleno: Todos los nodos internos tienen exactamente 2 hijos.

Árbol Binario Perfecto: Todos los niveles están completamente llenos. Tiene $2^{h+1} - 1$ nodos, donde h es la altura.

Árbol Binario Degenerado: Cada nodo tiene solo un hijo (similar a una lista enlazada).

3.2.3. Propiedades

- Número máximo de nodos en nivel i : 2^i
- Número máximo de nodos en árbol de altura h : $2^{h+1} - 1$
- Altura mínima con n nodos: $\lceil \log_2(n + 1) \rceil - 1$

3.3. Recorridos de Árboles Binarios

3.3.1. Pre-orden (Raíz-Izquierda-Derecha)

Algorithm 1 Recorrido Pre-orden

```

procedure PREORDEN(nodo)
  if nodo  $\neq$  null then
    Visitar(nodo)
    PreOrden(nodo.izquierdo)
    PreOrden(nodo.derecho)
  end if
end procedure

```

3.3.2. In-orden (Izquierda-Raíz-Derecha)

Algorithm 2 Recorrido In-orden

```

procedure INORDEN(nodo)
  if nodo  $\neq$  null then
    InOrden(nodo.izquierdo)
    Visitar(nodo)
    InOrden(nodo.derecho)
  end if
end procedure

```

3.3.3. Post-orden (Izquierda-Derecha-Raíz)

Algorithm 3 Recorrido Post-orden

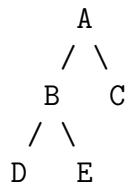
```

procedure POSTORDEN(nodo)
  if nodo  $\neq$  null then
    PostOrden(nodo.izquierdo)
    PostOrden(nodo.derecho)
    Visitar(nodo)
  end if
end procedure

```

3.3.4. Ejemplo de Recorridos

Para el árbol:



- **Pre-orden:** A, B, D, E, C
- **In-orden:** D, B, E, A, C
- **Post-orden:** D, E, B, C, A

3.4. Árboles Balanceados

3.4.1. Definición

Un árbol está **balanceado** si para cada nodo, la diferencia de altura entre su subárbol izquierdo y derecho es como máximo 1.

3.4.2. Factor de Balance

$$FB(nodo) = altura(subárbol_izquierdo) - altura(subárbol_derecho)$$

Un árbol está balanceado si: $|FB(nodo)| \leq 1$ para todo nodo.

3.5. Árboles N-arios

3.5.1. Definición

Árbol donde cada nodo puede tener un número arbitrario de hijos.

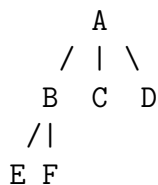
3.5.2. Transformada de Knuth

Técnica para convertir un árbol N-ario en un árbol binario:

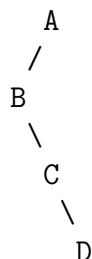
- El **primer hijo** de un nodo se convierte en su **hijo izquierdo**
- El **siguiente hermano** se convierte en su **hijo derecho**

Ejemplo:

Árbol N-ario:



Árbol Binario (Knuth):



/
E
\
F

4. Unidad 4: Acceso de Datos

4.1. Tablas de Símbolos

4.1.1. Definición

Estructura de datos que permite almacenar pares (*clave*, *valor*) con las operaciones:

- `Insert(clave, valor)`
- `Search(clave) → valor`
- `Delete(clave)`

4.1.2. Costos de Acceso

- **Búsqueda secuencial:** $O(n)$
- **Búsqueda binaria (arreglo ordenado):** $O(\log n)$
- **Árbol de búsqueda:** $O(h)$ donde h es la altura
- **Tabla Hash:** $O(1)$ promedio

4.2. Árboles de Búsqueda Binaria (ABB)

4.2.1. Definición

Árbol binario donde para cada nodo:

- Todos los valores del subárbol izquierdo son **menores**
- Todos los valores del subárbol derecho son **mayores**

4.2.2. Operaciones

Algorithm 4 Búsqueda en ABB

```
procedure BUSCAR(nodo, clave)
  if nodo = null then
    return null
  end if
  if clave = nodo.valor then
    return nodo
  else if clave < nodo.valor then
    return Buscar(nodo.izquierdo, clave)
  else
    return Buscar(nodo.derecho, clave)
  end if
end procedure
```

Búsqueda:

Inserción: Similar a búsqueda, pero se agrega el nodo en la posición correspondiente.

Eliminación: Casos:

1. Nodo hoja: Se elimina directamente
2. Nodo con un hijo: Se reemplaza por su hijo
3. Nodo con dos hijos: Se reemplaza por su predecesor o sucesor in-orden

4.3. Árboles AVL

4.3.1. Definición

ABB autobalanceado donde el factor de balance de cada nodo es -1 , 0 o 1 .

4.3.2. Rotaciones

Para mantener el balance después de inserciones/eliminaciones:

Rotación Simple a la Derecha: Usada cuando el subárbol izquierdo del hijo izquierdo causa desbalance.

Rotación Simple a la Izquierda: Usada cuando el subárbol derecho del hijo derecho causa desbalance.

Rotación Doble Izquierda-Derecha: Usada cuando el subárbol derecho del hijo izquierdo causa desbalance.

Rotación Doble Derecha-Izquierda: Usada cuando el subárbol izquierdo del hijo derecho causa desbalance.

4.4. Hashing

4.4.1. Concepto

Técnica que utiliza una función hash para mapear claves a posiciones en una tabla.

$$h : K \rightarrow \{0, 1, \dots, m - 1\}$$

donde K es el conjunto de claves y m es el tamaño de la tabla.

4.4.2. Funciones de Transformación

Método de División:

$$h(k) = k \bmod m$$

donde m es generalmente un número primo.

Método de Multiplicación:

$$h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor$$

donde $0 < A < 1$ (comúnmente $A \approx 0,618$).

Para Cadenas de Caracteres: Convertir cada carácter a su código ASCII y aplicar una función:

$$h(s) = \left(\sum_{i=0}^{n-1} ASCII(s[i]) \cdot 31^i \right) \bmod m$$

Ejemplo con método de división:

Clave: "Hola"

Conversión: $H(72) + o(111) + l(108) + a(97) = 388$

Con $m = 50$: $h(\text{"Hola"}) = 388 \bmod 50 = 38$

4.4.3. Colisiones

Definición: Ocurre cuando dos claves diferentes producen el mismo valor hash: $h(k_1) = h(k_2)$ con $k_1 \neq k_2$.

Resolución por Encadenamiento: Cada posición de la tabla contiene una lista enlazada con todos los elementos que tienen ese hash.

Resolución por Direccionamiento Abierto:

- **Prueba Lineal:** $h'(k, i) = (h(k) + i) \bmod m$
- **Prueba Cuadrática:** $h'(k, i) = (h(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$
- **Hash Doble:** $h'(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$

4.5. Árboles B**4.5.1. Definición**

Árbol de búsqueda N-ario balanceado optimizado para acceso a disco, donde:

- Cada nodo puede contener múltiples claves
- Todas las hojas están al mismo nivel
- Se mantiene un orden de t (grado mínimo)

4.5.2. Propiedades del Árbol B de orden t :

- Cada nodo (excepto la raíz) tiene entre $t - 1$ y $2t - 1$ claves
- Cada nodo interno (excepto la raíz) tiene entre t y $2t$ hijos
- La raíz tiene al menos 1 clave
- Todas las hojas están al mismo nivel

5. Ejemplos Prácticos Adicionales

5.1. Ejemplo: Operaciones con Pila y Cola

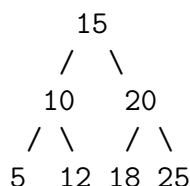
Situación: Dada la lista: [12, 7, 18, 5, 9, 14]

Proceso:

- Elementos pares (12, 18, 14) $\rightarrow$ Pila: [12, 18, 14] (tope en 14)
- Elementos impares (7, 5, 9) $\rightarrow$ Cola: [7, 5, 9]
- Promedio de la pila: $(12 + 18 + 14)/3 = 14,67$

5.2. Ejemplo: Recorrido de Árbol

Para el árbol:



- **Tipo:** Árbol binario de búsqueda
- **Completo:** Sí
- **Altura:** 2
- **Balanceado:** Sí (FB = 0 para todos los nodos)
- **Pre-orden:** 15, 10, 5, 12, 20, 18, 25
- **In-orden:** 5, 10, 12, 15, 18, 20, 25
- **Post-orden:** 5, 12, 10, 18, 25, 20, 15

5.3. Ejemplo: Hashing con Método de División

Claves: “Ana”, “Juan”, “María”, “Pedro”

Tabla: $m = 50$

Conversión ASCII y cálculo:

$$\text{Ana} = 65 + 110 + 97 = 272 \rightarrow 272 \text{ mód } 50 = 22$$

$$\text{Juan} = 74 + 117 + 97 + 110 = 398 \rightarrow 398 \text{ mód } 50 = 48$$

$$\text{María} = 77 + 97 + 114 + 237 + 97 = 622 \rightarrow 622 \text{ mód } 50 = 22$$

$$\text{Pedro} = 80 + 101 + 100 + 114 + 111 = 506 \rightarrow 506 \text{ mód } 50 = 6$$

Nota: “Ana” y “María” tienen colisión en la posición 22.

6. Algoritmos Fundamentales

6.1. Algoritmo de Recorrido de Lista con Pila y Cola

Algorithm 5 Procesar Lista en Pila y Cola

```
procedure PROCESARLISTA(lista)
  pila  $\leftarrow$  nueva Pila()
  cola  $\leftarrow$  nueva Cola()
  suma_pila  $\leftarrow$  0
  contador_pila  $\leftarrow$  0
  for cada elemento en lista do
    if elemento mod 2 = 0 then                                      $\triangleright$  Es par
      pila.push(elemento)
      suma_pila  $\leftarrow$  suma_pila + elemento
      contador_pila  $\leftarrow$  contador_pila + 1
    else                                                            $\triangleright$  Es impar
      cola.enqueue(elemento)
    end if
  end for
  Mostrar(pila)
  Mostrar(cola)
  if contador_pila  $\neq$  0 then
    promedio  $\leftarrow$  suma_pila / contador_pila
    Mostrar("Promedio de pila:", promedio)
  end if
end procedure
```

6.2. Verificación de Balance en Árbol

Algorithm 6 Verificar si un árbol está balanceado

```
procedure CALCULARALTURA(nodo)
  if nodo = null then
    return -1
  end if
  return 1 +  $\max(\text{CalcularAltura}(\text{nodo.izq}), \text{CalcularAltura}(\text{nodo.der}))$ 
end procedure
procedure ESTABALANCEADO(nodo)
  if nodo = null then
    return true
  end if
  altura_izq  $\leftarrow$  CalcularAltura(nodo.izquierdo)
  altura_der  $\leftarrow$  CalcularAltura(nodo.derecho)
  factor_balance  $\leftarrow$  altura_izq - altura_der
  if  $|\text{factor\_balance}| > 1$  then
    return false
  end if
  return EstaBalanceado(nodo.izq) AND EstaBalanceado(nodo.der)
end procedure
```

7. Complejidades Temporales - Resumen

Estructura	Búsqueda	Inserción	Eliminación
Array (desordenado)	$O(n)$	$O(1)$	$O(n)$
Array (ordenado)	$O(\log n)$	$O(n)$	$O(n)$
Lista Enlazada	$O(n)$	$O(1)^*$	$O(1)^*$
Pila	-	$O(1)$	$O(1)$
Cola	-	$O(1)$	$O(1)$
ABB (balanceado)	$O(\log n)$	$O(\log n)$	$O(\log n)$
ABB (degenerado)	$O(n)$	$O(n)$	$O(n)$
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$
Hash (ideal)	$O(1)$	$O(1)$	$O(1)$
Árbol B	$O(\log n)$	$O(\log n)$	$O(\log n)$

*Si se tiene el puntero al nodo

8. Consejos para el Trabajo Integrador

8.1. Análisis de Grafos

1. Identificar vértices y aristas
2. Verificar dirección de las aristas
3. Comprobar conectividad (¿hay camino entre todos los pares?)
4. Verificar si hay pesos en las aristas
5. Construir la matriz siguiendo el orden de los vértices

8.2. Trabajar con Pilas y Colas

1. Identificar claramente el criterio de separación
2. Mantener el orden LIFO para pilas (último que entra, primero que sale)
3. Mantener el orden FIFO para colas (primero que entra, primero que sale)
4. Para promedios: sumar todos los elementos y dividir por la cantidad

8.3. Análisis de Árboles

1. Identificar la raíz y los nodos hoja
2. Calcular la altura contando niveles desde 0
3. Para verificar balance: calcular altura de subárboles izquierdo y derecho
4. Para recorridos: seguir estrictamente el orden indicado (pre/in/post)
5. Un árbol es completo si todos los niveles están llenos excepto posiblemente el último

8.4. Hashing

1. Convertir cada carácter a su valor ASCII
2. Sumar todos los valores
3. Aplicar módulo m (el tamaño de la tabla)
4. Documentar claramente los cálculos
5. Identificar posibles colisiones (mismo resultado para claves diferentes)

*Este resumen cubre todos los conceptos teóricos necesarios
para resolver el Trabajo Final Integrador de Introducción a la Estructura de Datos.*

¡Éxitos en el examen!